

Emergency Social Network (ESN) System

This is a web-based emergency communication and coordination platform designed to improve communication, coordination, and information sharing among community members during emergencies.

The ESN system enables real-time communication and coordination during emergencies, allowing citizens, coordinators, and administrators to interact through a secure online platform.

User Roles & Hierarchy

Citizens: Regular community members who can receive alerts and communicate

Coordinators: Community leaders who can post announcements.

Administrators: System-wide access to manage users, communities, and oversee the entire platform

Geographic Structure

Regions → Districts → Communities

Users are assigned to specific geographic locations

MVC (Model-View-Controller) is a software architectural pattern that separates an application mainly into three interconnected components: the Model (data and business logic), the View (user interface), and the Controller (handles user input and routes requests). This separation of concerns improves organization, makes code easier to maintain, and allows for simultaneous development of different parts.

Separation of concerns (SoC) is a software design principle that divides a program into distinct sections, with each section handling a single, specific "concern" or functionality.

This modular approach improves code clarity, maintainability, testability, and reusability by keeping different parts of the system independent, such as separating user interface logic from business logic.

USE CASE 1: ADMINISTER USER PROFILE

At-least-One-Administrator Rule:

- **Implemented in:** `src/utils/initAdmin.js`
- **Enforcement:** The `validateAdminCount()` function prevents deletion of the last admin
- **Applied in:** User deletion, role changes, and status changes

Initial-Administrator Rule:

- **Implemented in:** `src/utils/initAdmin.js`
- **Default Admin:** Username: `ESNAdmin`, Password: `admin`
- **Auto-creation:** System automatically creates this admin on startup
- **Verification:** System verifies `ESNAdmin` exists after initialization

User Profile Administration Features:

1. Account Status Management:

- **Location:** `src/controllers/userController.js`
- **Function:** `updateUserStatus()` - Switch users between Active/Inactive
- **UI:** Admin panel with status toggle in `src/views/admin-users.ejs`

2. Privilege Level Management:

- **Location:** src/controllers/userController.js
- **Function:** updateUserByAdmin() - Change user roles (Admin/ Coordinator/ Citizen)
- **Default:** New accounts default to 'citizen' role
- **UI:** Role selection dropdown in admin interface

3. Username Changes:

- **Location:** src/controllers/userController.js
- **Validation:** Username uniqueness checking
- **UI:** Username field in edit user modal

4. Password Changes:

- **Location:** src/controllers/userController.js
- **Function:** changeUserPassword() - Admin can change any user's password
- **Security:** Password hashing with bcrypt
- **UI:** Dedicated password change modal

Active/Inactive Rule Implementation:

- **User Model:** src/models/User.js - isActive field with default true
- **Session Management:** Inactive users are logged out automatically
- **Visibility:** Inactive user content is hidden from other users
- **Admin Access:** Admins can still see inactive user profiles for administration

USE CASE 2: POST ANNOUNCEMENT

Coordinator-Only Access:

- **Role Restriction:** `src/controllers/announcementController.js`
- **Middleware:** Only coordinators and admins can post announcements
- **Route Protection:** `src/routes/announcementRoutes.js`

Announcement Creation Flow:

1. Coordinator Access Control:

- **Location:** `src/routes/announcementRoutes.js`
- **Route:** `/announcements/manage` - Coordinator-only page
- **Verification:** Role check ensures only coordinators can access

2. Announcement Creation Interface:

- **UI:** `src/views/coordinator-announcements.ejs`
- **Features:** Title, message, type, priority, attachments, pinning
- **Emergency Support:** Special fields for emergency announcements

3. Backend Processing:

- **Controller:** `src/controllers/announcementController.js`
- **Function:** `createAnnouncement()` - Handles announcement creation
- **Features:** File attachments, emergency classification, real-time notifications

Announcement Display:

- **Public Visibility:** All community members can see announcements
- **Real-time Updates:** Socket.io integration for live updates
- **Search & Filter:** Advanced filtering by type, priority, date
- **Pagination:** Efficient handling of large announcement lists

Alternative Flow (A1):

- **Empty State:** System handles cases with no existing announcements
- **UI:** Shows "No announcements" message when list is empty
- **Creation Space:** Always provides space to create new announcements

ADDITIONAL FEATURES BEYOND REQUIREMENTS

Enhanced User Management:

- **Bulk Operations:** Export users, status changes
- **Advanced Filtering:** Search by role, status, location, online status
- **Geographic Management:** Region/district-based user address

Advanced Announcement Features:

- **Emergency Classification:** Medical, fire, security, natural disaster types
- **Severity Levels:** Low, medium, high, critical priority system
- **File Attachments:** Support for documents, images, PDFs
- **Forwarding System:** Forward announcements to public/private chats
- **View Tracking:** Monitor views for the announcements

Security & Access Control:

- **Role-Based Access:** Granular permissions for different user types
- **Session Management:** Secure session handling with automatic logout
- **File Upload Security:** Secure file handling with type validation
- **Input Validation:** Comprehensive validation for all user inputs

Implementation Summary:

Use Case	Requirement
UC1	At-least-One-Admin Rule

UC1	Initial-Admin Rule
UC1	Account Status Management
UC1	Privilege Level Changes
UC1	Username Changes
UC1	Password Changes
UC2	Coordinator-Only Access
UC2	Announcement Creation
UC2	Public Display
UC2	Alternative Flow (A1)